

Computer Graphics

Lecture 09

2D Viewing & Clipping

Topics

- **Viewing Pipeline:** describes a series of transformation to display world data on a device
- **Viewing Transformation:** how it transforms world data for a device
- **Viewing Coordinate Mapping:**
- **Clipping Algorithms:**



Viewing Pipeline

Suppose you are watching a hill from your roof. Only rectangular block view of scene can be seen through eyes. This block is called **window**.

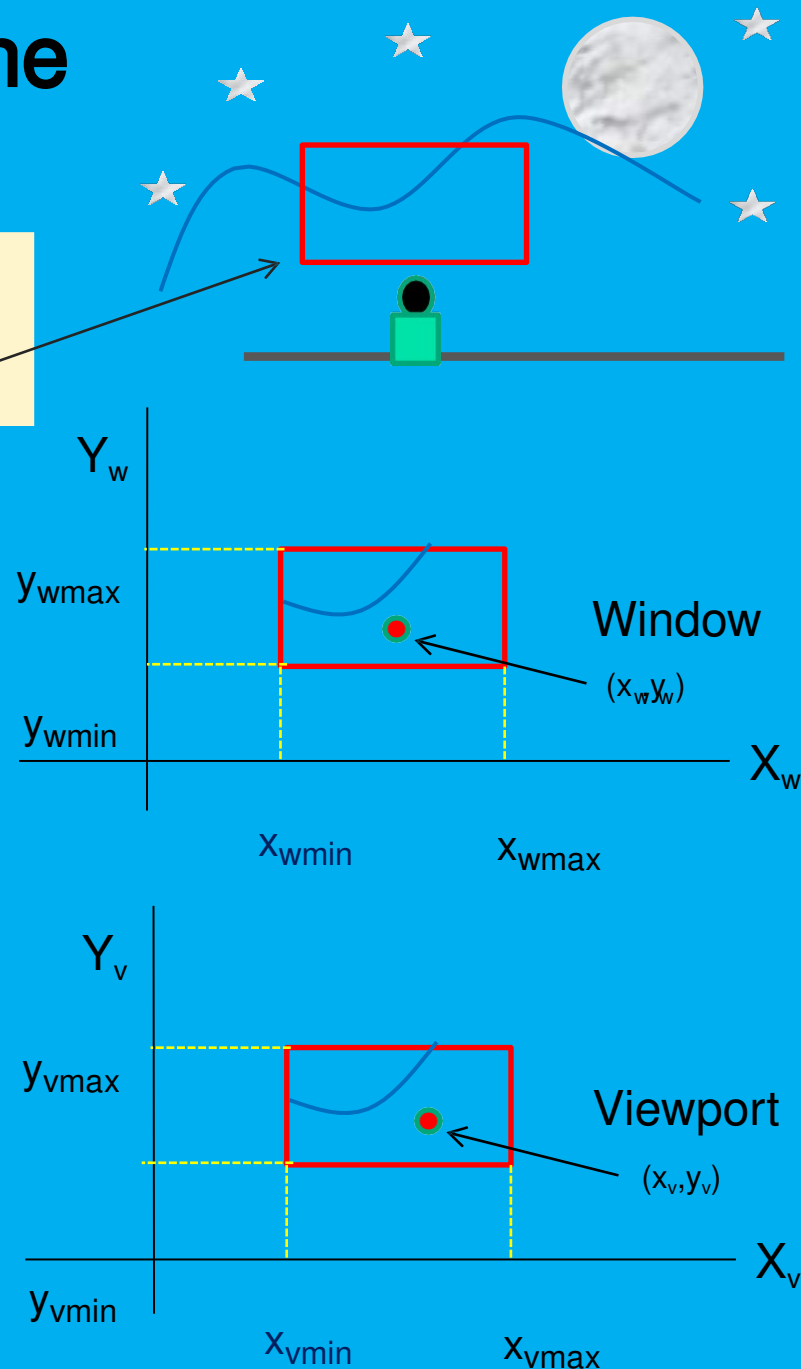
The window can be represented by world coordinate system.

Now we need to present this window on a device called **viewport**.



Window to Viewport Transformation

Now how a point in window is to be represented in viewport?



Viewing Transformation

Process of viewing transformation:

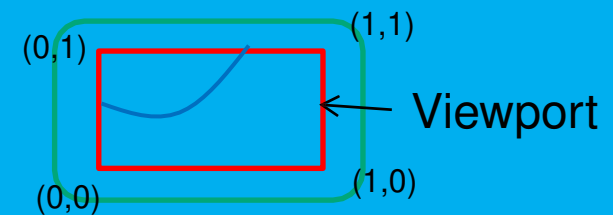
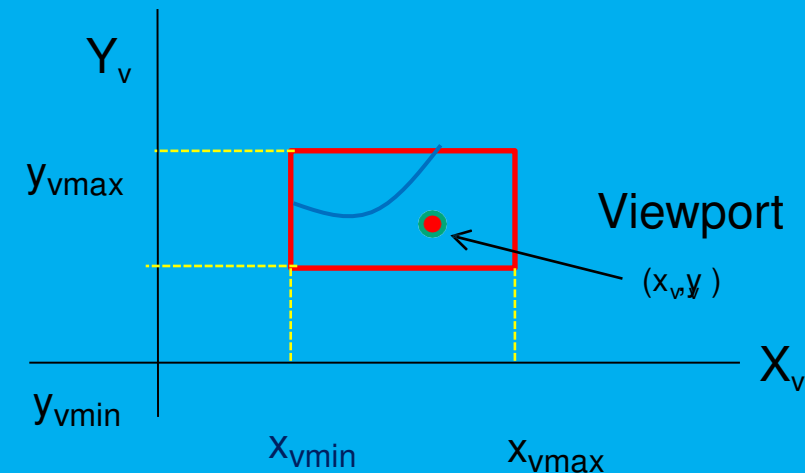
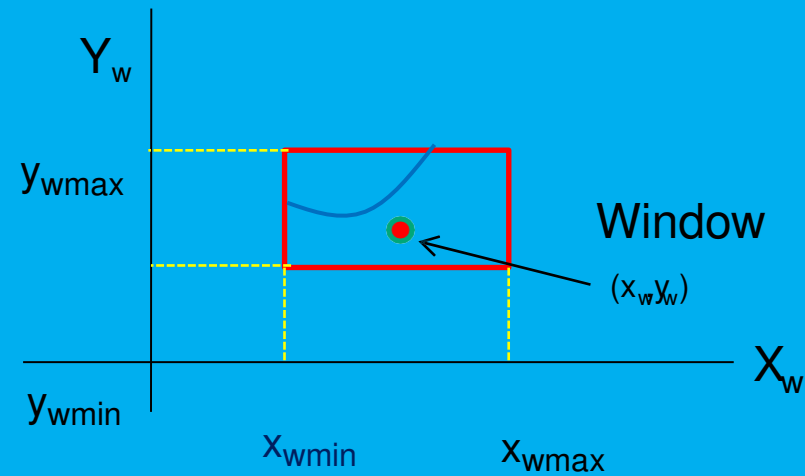
1. Normalization Transformation
2. Work Station Transformation

Normalization Transformation:

As image defined as set of pixels, it displays in different size in different devices

Suppose an image defined in low resolution device shrinks in size in high resolution device because a number of pixels in high resolution device takes less space than in low resolution device.

To make it device independent we need Normalize transformation. It transfer window Coordinate to normalize device coordinate makes it device independent. It consider 1 unit in both x and y axis.



World
Coordinat $\xrightarrow{\text{Normalization}}$ Normalize
Coordinat $\left[\begin{smallmatrix} 0 & -1 \end{smallmatrix} \right]$

Device

Viewing Transformation

Normalization Transformation:

$$x = x_n \cdot x_w$$

$$y = y_n \cdot Y_w$$

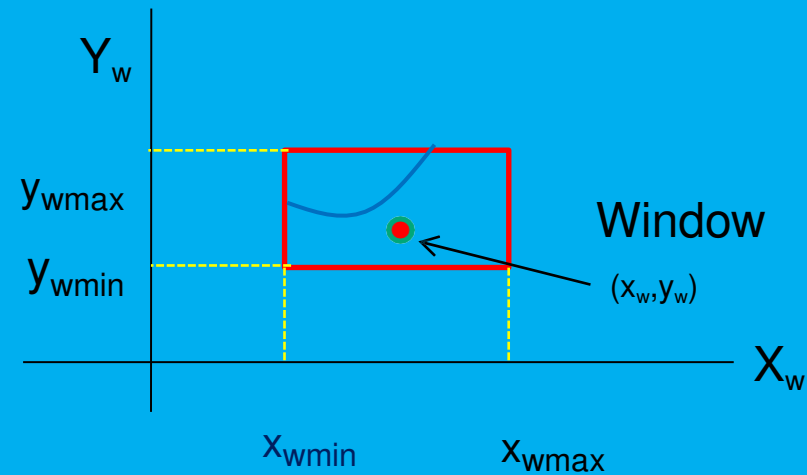
Here

(x, y) = Actual device coordinate

(x_w, Y_w) = Window coordinate

(x_n, Y_n) = Normalize coordinate

(scaling)



Work Station Transformation:

Here world coordinate transforms to Normalize coordinate and then apply work station to get device coordinate

World
Coordinat

Normalization

Normalize
Coordinate

Work Station

Device
Coordinat

e

We can write

$$V = N.W$$

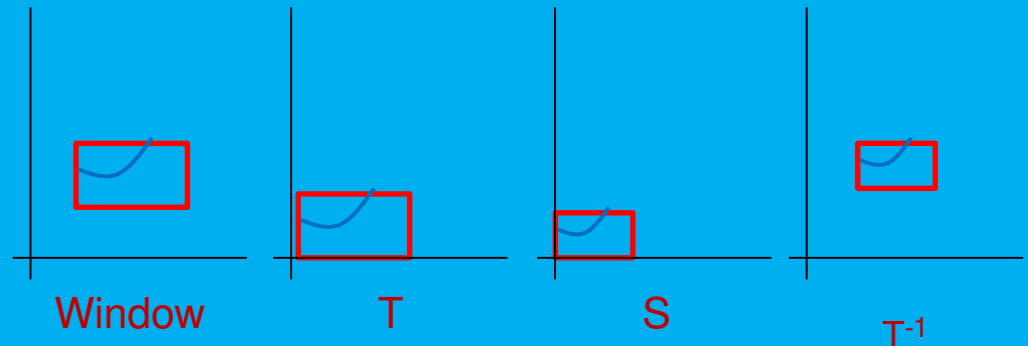
$$W = TST^{-1}$$

Here,

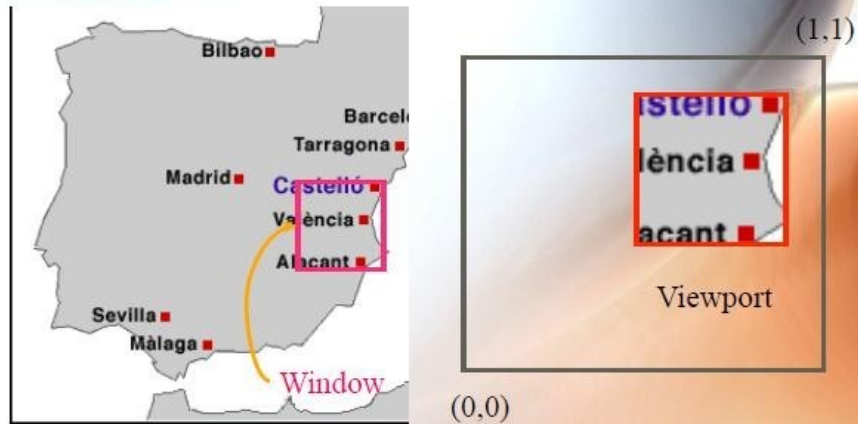
T means translate image to origin

S scaling according to device

T^{-1} inverse translate image



Viewing Transformation (World to NDC)

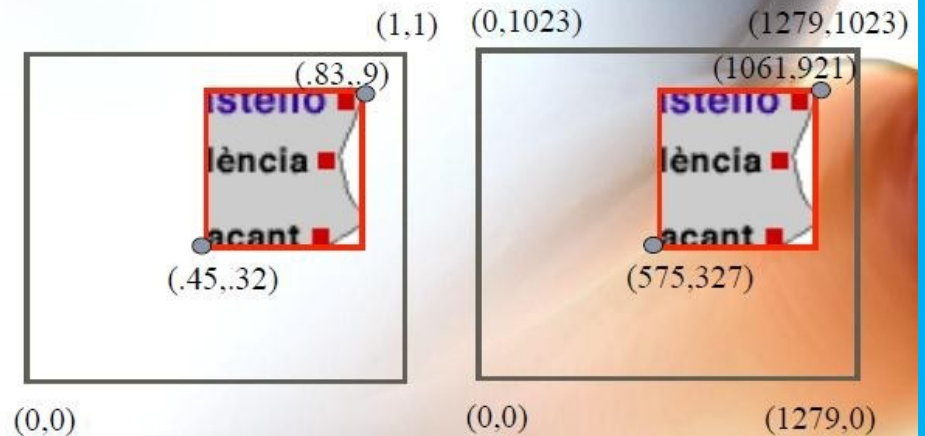


World Coordinates

Normalized Device Coordinates

Transformation Process

NDC to Screen



Normalized Device Coordinates

Screen Coordinates

Viewing Transformation

We can write

$$V = N.W$$

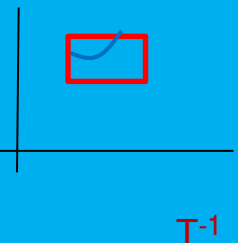
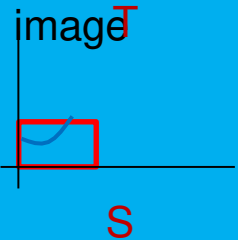
$$W = TST^{-1}$$

Here,
 T means
 translate
 image to

origin

S scaling
 according to
 device

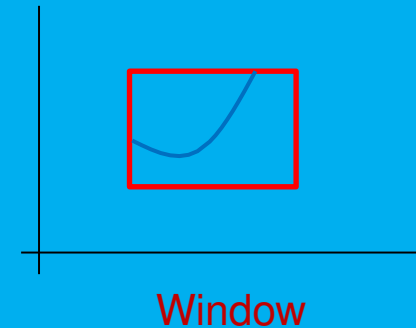
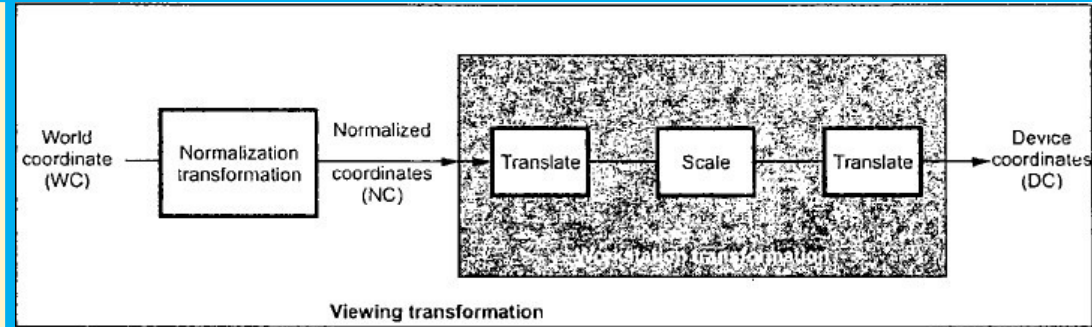
T^{-1} inverse
 translate
 image



$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

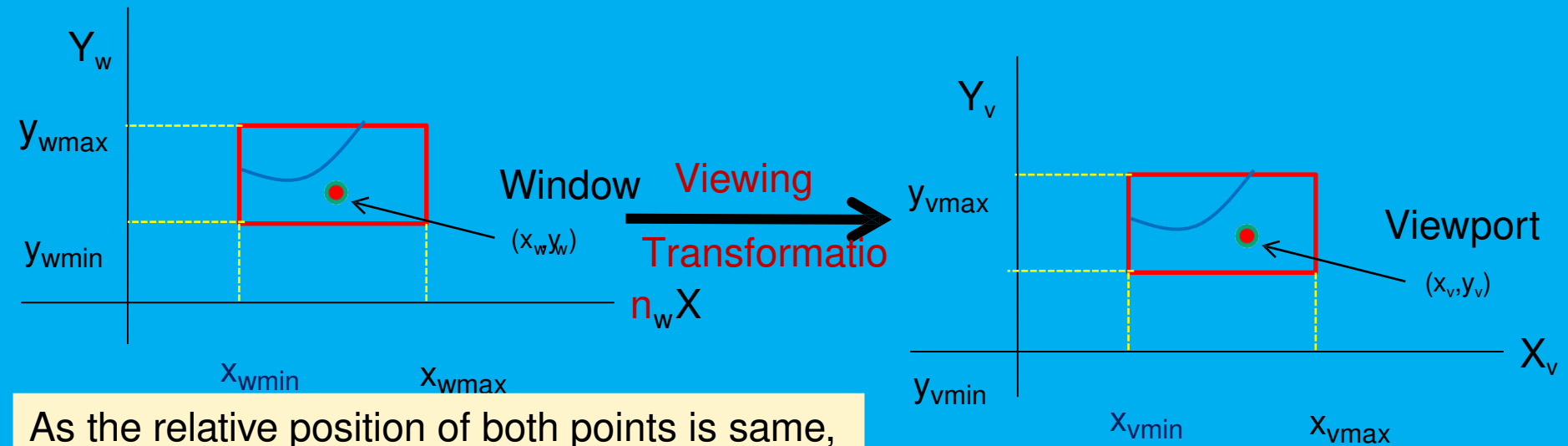
$$^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



$$^{-1} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Window to Viewport Mapping

Let a window point (x_w, y_w) mapping to viewport (x_v, y_v)



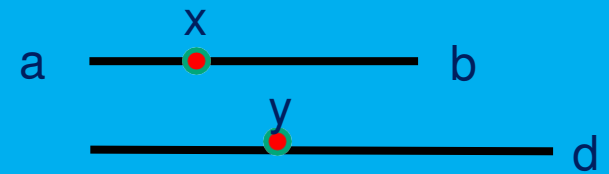
As the relative position of both points is same, we can write

$$\frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}} = \frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}}$$

$$= \frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}} (x_{wmax} - x_{wmin}) + x_{wmin}$$

$$= \frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}} (x_{wmax} - x_{wmin}) + x_{wmin}$$

S_x and S_y are scaling factors

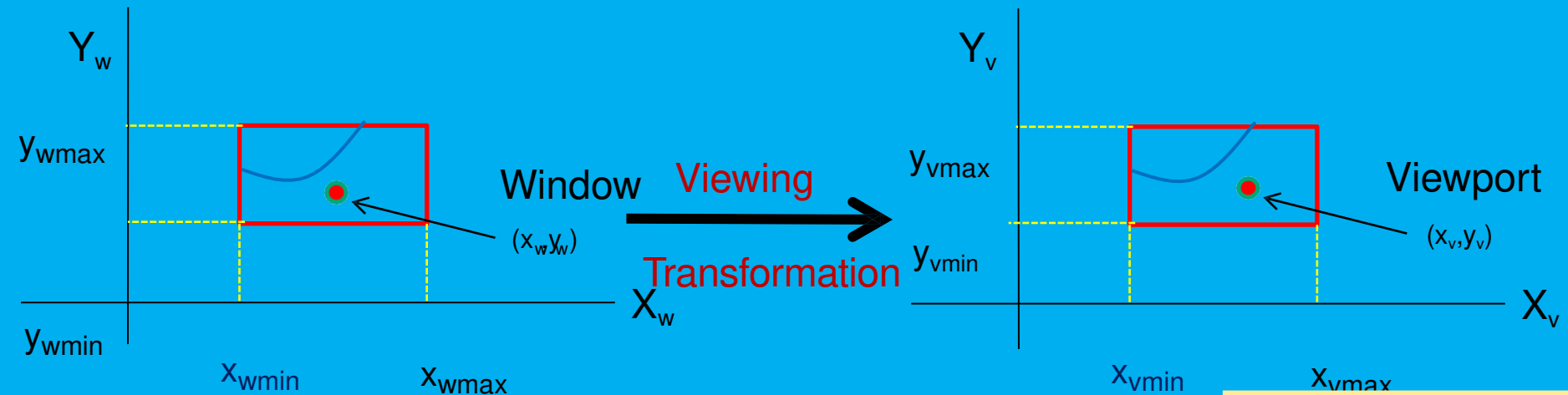


$$\frac{x - a}{b - a} = \frac{y - c}{d - c}$$

$$= \frac{y - c}{d - c} (b - a) + a$$

Window to Viewport Mapping

Let a window point (x_w, y_w) mapping to viewport (x_v, y_v)



$$= 20$$

$$= 80$$

$$= 40$$

$$= 80$$

$$(\quad , \quad) = (30,$$

$$80)$$

$$= \quad + \quad (\quad -$$

$$= 30 + 0(5 \quad 30 \quad -)20 =$$

$$35$$

$$S = \frac{\quad - \quad}{\quad - \quad} = \frac{60 - 30}{80 - 20} = 0.5$$

$$S = \frac{\quad - \quad}{\quad - \quad} = \frac{60 - 40}{80 - 40} = 0.5$$

$$= \quad + \quad (\quad - \quad)$$

$$= 40 + 0(5 \quad 80 \quad -)40 =$$

$$60$$

$$=$$

$$30$$

$$=$$

$$60$$

$$=$$

$$40$$

$$=$$

$$60$$

$$(\quad , \quad)$$

$$= ?$$

Example on Workstation Transformation

Find the normalization transformation window to viewport, with window, lower left corner at (1, 1) and upper right corner at (3, 5) onto a viewport with lower left corner at (0, 0) and upper right corner at (1/2, 1/2).

Given : Coordinates for window

$$x_{w \min} = 1 \quad y_{w \min} = 1$$

$$x_{w \max} = 3 \quad y_{w \max} = 5$$

Coordinates for view port

$$x_{v \min} = 0 \quad y_{v \min} = 0$$

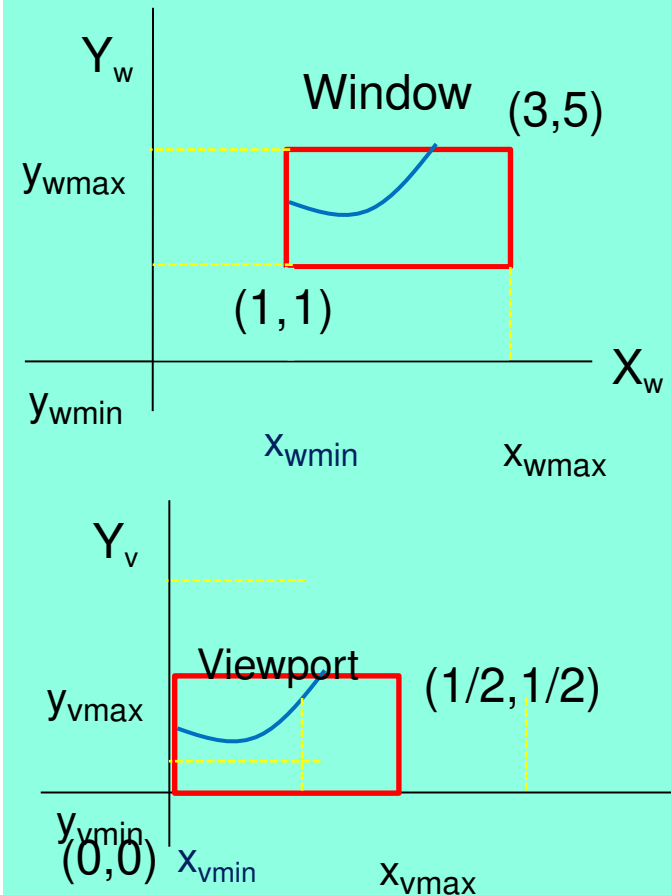
$$x_{v \max} = 1/2 = 0.5 \quad y_{v \max} = 1/2 = 0.5$$

We know that,

$$\begin{aligned} S_x &= \frac{x_{v \max} - x_{v \min}}{x_{w \max} - x_{w \min}} \\ &= \frac{0.5 - 0}{3 - 1} \\ &= 0.25 \end{aligned}$$

and

$$\begin{aligned} S_y &= \frac{y_{v \max} - y_{v \min}}{y_{w \max} - y_{w \min}} \\ &= \frac{0.5 - 0}{5 - 1} \\ &= 0.125 \end{aligned}$$



Example on Workstation Transformation

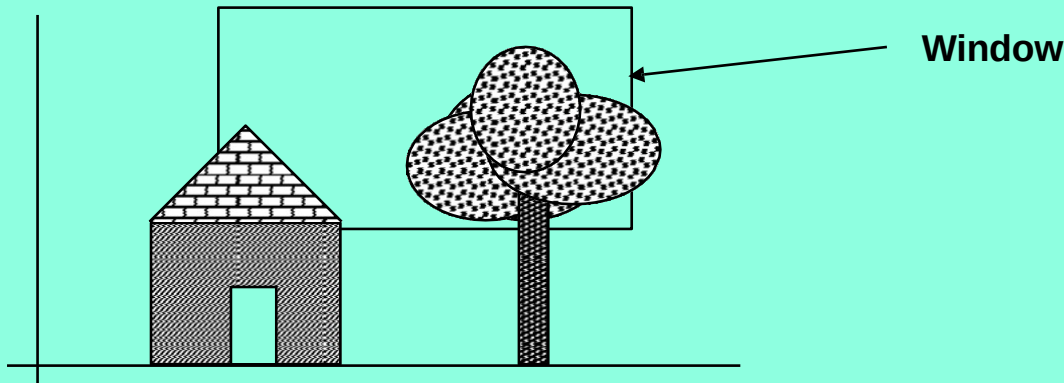
Find the normalization transformation window to viewport, with window, lower left corner at (1, 1) and upper right corner at (3, 5) onto a viewport with lower left corner at (0, 0) and upper right corner at (1/2, 1/2).

Given : Coordinates for window

We know that transformation matrix is given as

$$\begin{aligned} T \cdot S \cdot T^{-1} &= \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ x_{v \min} - x_{w \min} S_x & y_{v \min} - y_{w \min} S_y & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 0.125 & 0 \\ 0 - (1 \times 0.25) & 0 - (1 \times 0.125) & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 0.125 & 0 \\ -0.25 & -0.125 & 1 \end{bmatrix} \end{aligned}$$

Window & Clipping



Clipping: The procedure that identifies the portion of a picture That are either inside or outside of a specified region of space is referred to as Clipping. The region against which an object is to be clipped is called **a clip Window** or **clipping window**.

Types of Clipping: Discarding the portion outside the window

Types:

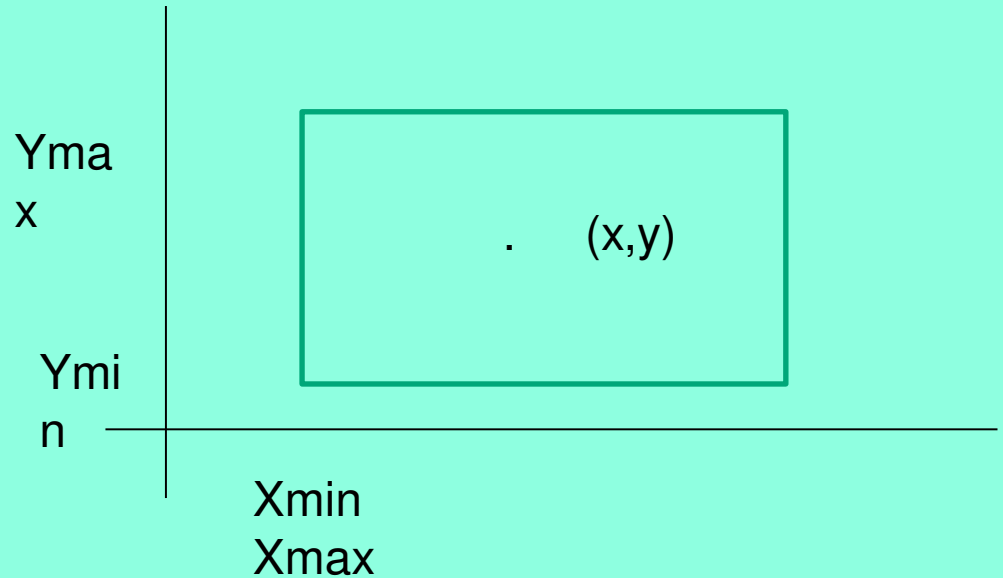
1. Point Clipping
2. Line Clipping
 - a) Cohen-Sutherland Algorithm
 - b) Cyrus-Beck Algorithm
 - c) Liang-Bersky Algorithm
3. Polygon Clipping
 - Sutherland-Hodgeman Algorithm
4. Curve Clipping
4. Text Clipping

Point Clipping

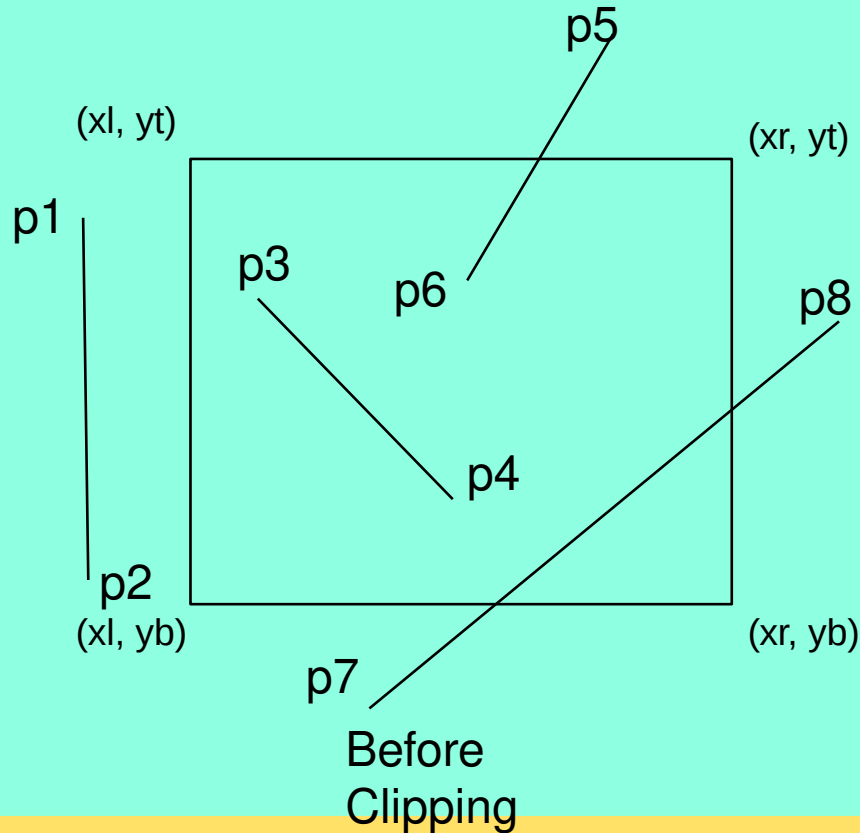
Define a window first

Conditions for clipping (x,y)

- a. $X_{min} \leq x \leq X_{max}$
- b. $Y_{min} \leq y \leq Y_{max}$



Line Clipping



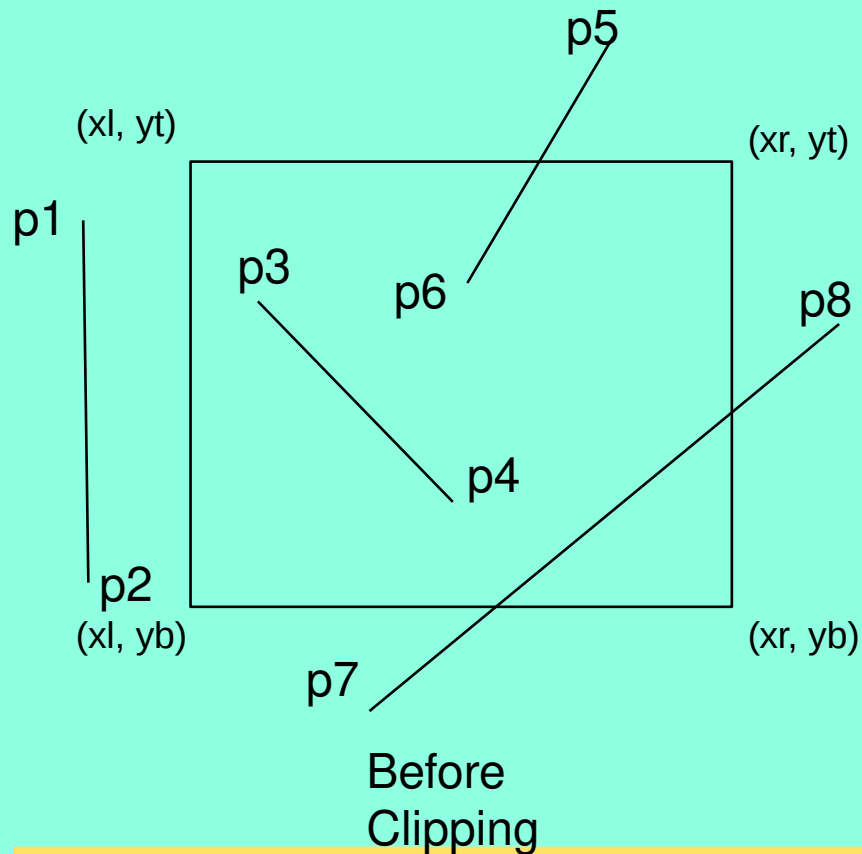
Conditions for clipping (x, y)

a. $X_{min} \leq x \leq X_{max}$

b. $Y_{min} \leq y \leq Y_{max}$

- A **line** is visible if both of its end points are in the window.

Line Clipping



$p1-p2$ □ Reject

$p3-p4$ □ Consider (Accept)

$p5-p6$ □ Clipping

Required $p7-p8$ □

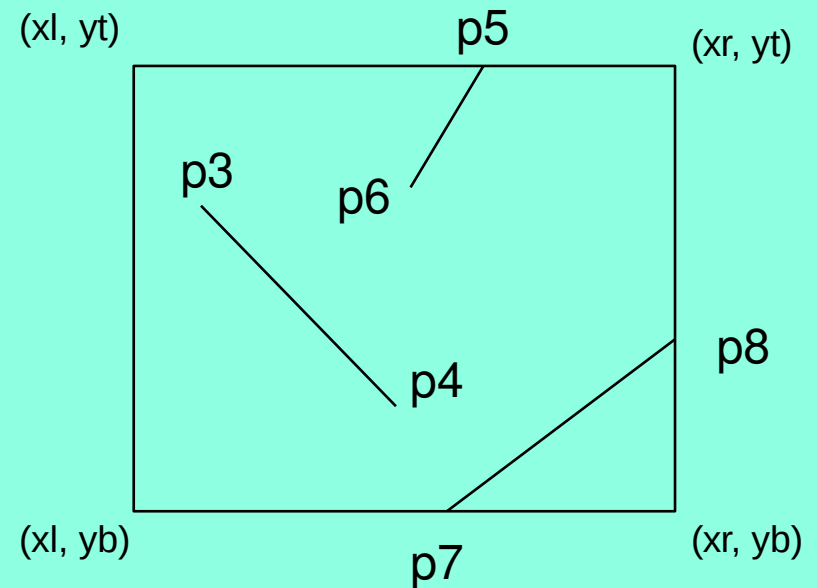
Clipping Required

A **point** is visible if

$$xl < X < xr$$

and

$$yb < Y < yt$$



After

Cohen-Sutherland Algorithm

- **Region Checks:** Trivially reject or accept lines and points.
- Fast for large windows (everything is inside) and for small windows (everything is outside).
- Each vertex is assigned a four-bit **outcode**.
 - Bit 1 <-- sign bit of $(y_t - Y)$ -- point is above window
 - Bit 2 <-- sign bit of $(Y - y_b)$ -- point is below window
 - Bit 3 <-- sign bit of $(x_r - X)$ -- point is to right of window
 - Bit 4 <-- sign bit of $(X - x_l)$ -- point is to left of window

Cohen-Sutherland Clipping

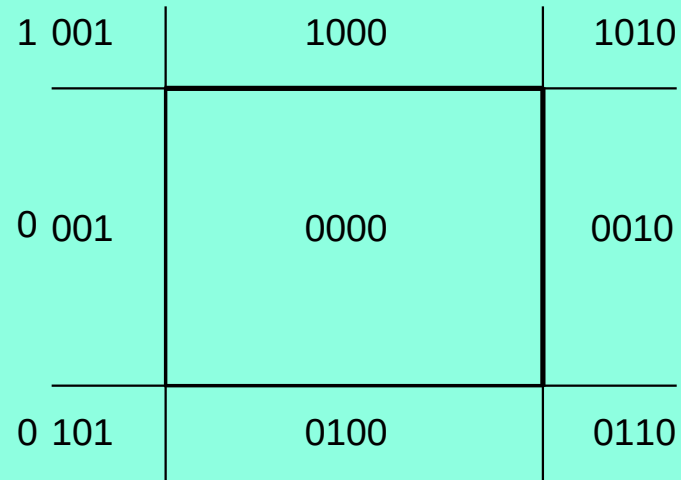
ABRL Code

Bit 1: Above

Bit 2: Below

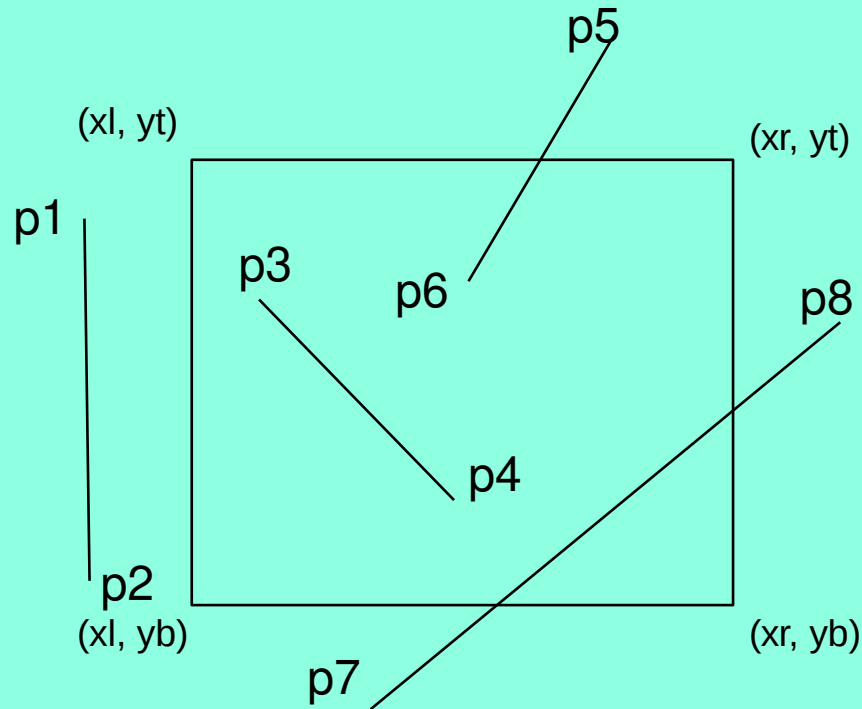
Bit 3: Right

Bit 4: Left



- A line can be trivially **accepted** if both endpoints have an outcode of 0000.
- A line can be trivially **rejected** if any corresponding bits in the two outcodes are both equal to 1. (This means that both endpoints are to the right, to the left, above, or below the window.) Can be computed with a logical AND.

Cohen-Sutherland Clipping



1 001	1000	1010
0 001	0000	0010
0 101	0100	0110

ABRL Code

P1 □ 0001 – Non Zero

P2 □ 0001 – Non

~~AND~~ □ 0001 – Non Zero

p1p2 is completely outside
REJET LINE

ABRL Code

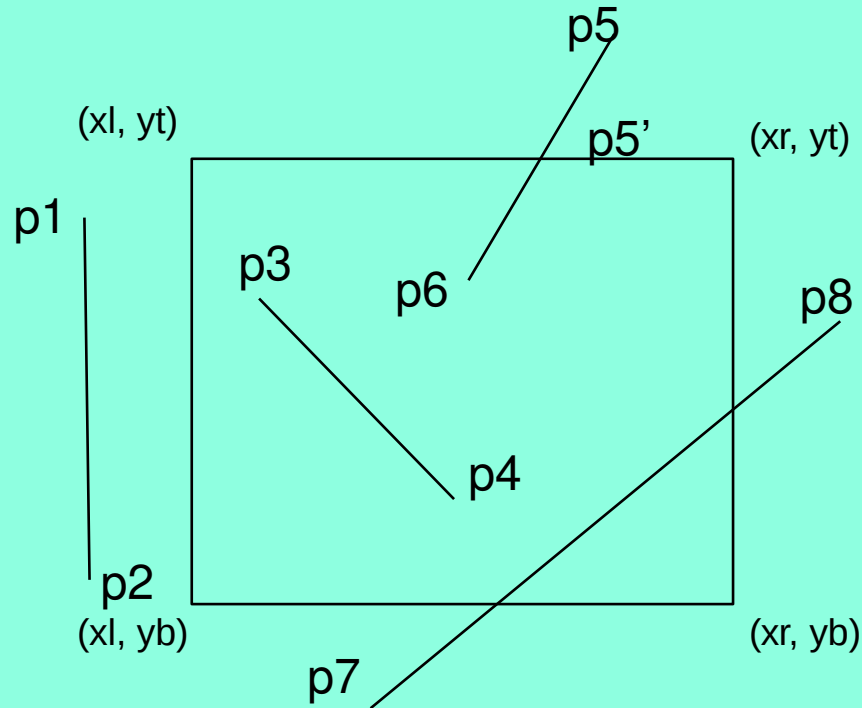
P3 □ 0000 –Zero

P4 □ 0000 –Zero

AND □ 0000 –Zero

p3p4 is completely inside
ACCEPT LINE

Cohen-Sutherland Clipping



1 001	1000	1010
0 001	0000	0010
0 101	0100	0110

ABRL Code

P5 □ 1000 – Non Zero

P6 □ 0000 – Zero

AND □ 0000 – Zero

Clipping is Required

**Let p5' is intersection
on window**

P5' □ 0000 – Zero

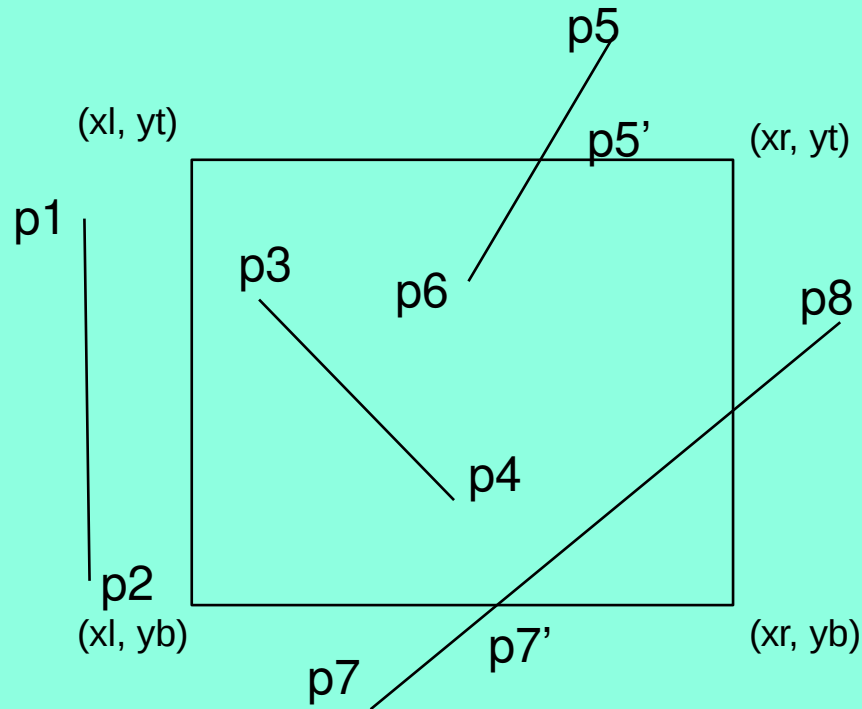
P6 □ 0000 – Zero

AND □ 0000 – Zero

P5'p6 is inside

p5p5' is outside

Cohen-Sutherland Clipping



1 001	1000	1010
0 001	0000	0010
0 101	0100	0110

ABRL Code

P7 □ 0100 – Non Zero

P8 □ 0010 – Non Zero

AND □ 0000 – Zero

Clipping is Required

**Let p7' is intersection
on window**

p7p7' □ clipped

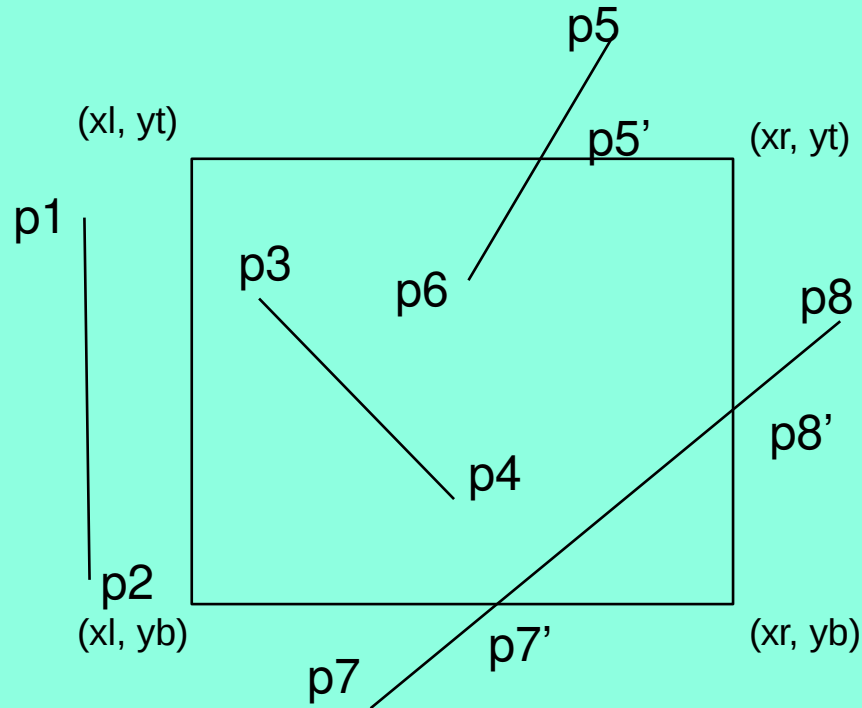
**Let p7' is intersection
on window**

P7' □ 0000 – Zero

P8 □ 0010 – Non
Zero

AND □ 0000 – Zero
Clipping is Required

Cohen-Sutherland Clipping



1 001	1000	1010
0 001	0000	0010
0 101	0100	0110

Let $p8'$ is intersection
on window
 $p8p8' \square$ clipped

$P8' \square 0000 -$

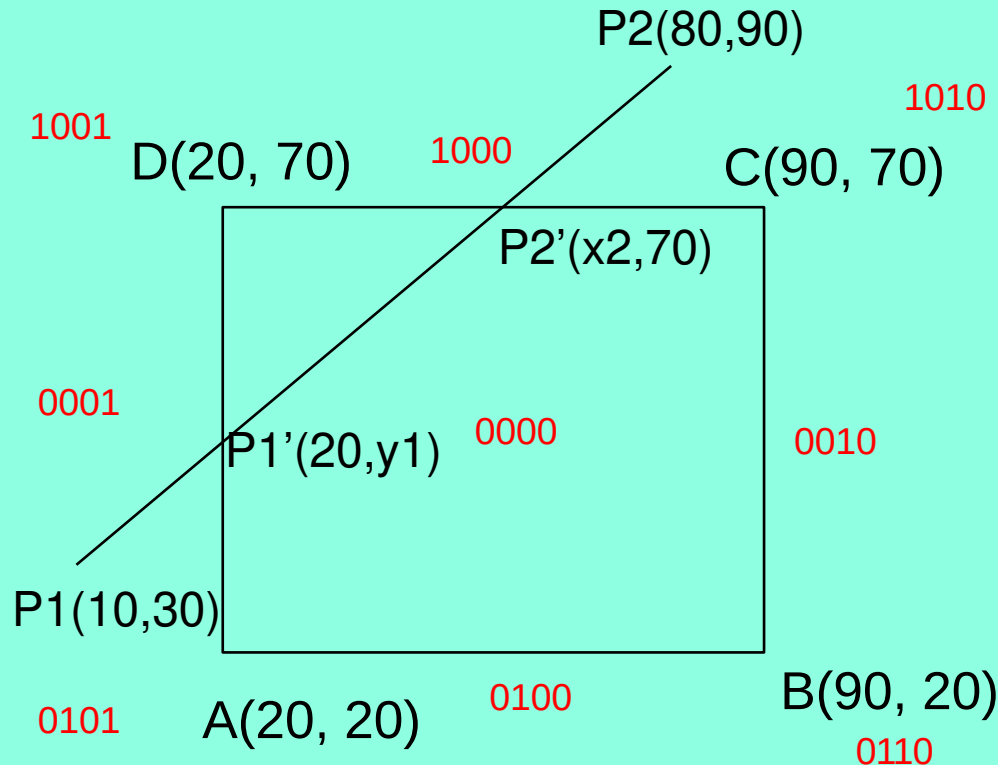
Zero $P7' \square$

$AND \square 0000 - \text{Zero}$

$P8'p7'$ is completely inside

Cohen-Sutherland Clipping

Prob: Let ABCD be the rectangular window with A(20,20), B(90,20), C(90,70), D(20,70). Find the codes for endpoints and use Cohen-Sutherland algorithm to clip lines with P1(10,30) and P2(80,90).



Step 1: Find

P1	0001
P2	1000

Step 2: Check Visibility of

P1P2	0001 – Non
Zero	P2 1000 –
AND	0000 – Zero

**P1P2 is partially inside
(trivially accepted)**

Step 3: Find intersection Points

$$m = (90 - 30) / (80 - 10) = 0.85$$

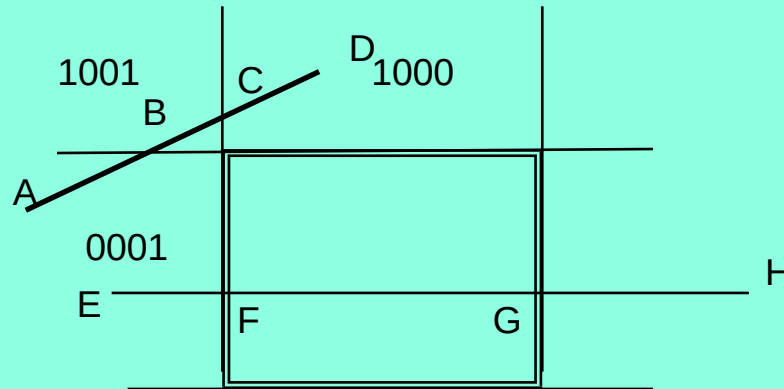
$$(y1 - 30) / (20 - 10) = 0.85 \Rightarrow y1 = 39$$

$$(90 - 70) / (80 - x2) = 0.85 \Rightarrow x2 = 57$$

Clipping Lines not Accepted or Rejected

In the case where a line can be neither trivially accepted nor rejected, the algorithm uses a “divide and conquer” method.

Example:



- Line AD:
- 1) Test outcodes of A (0001) and D (1000) . Here A and B =0000, accept or reject?
 - 2) Calculate intersection point B, which is conceptually on the window side of the dividing line. Here A and B=1000 which is non-zero, so it is outside the window. Then B and D=1000, so it is also nonzero hence outside.

Polygon Clipping

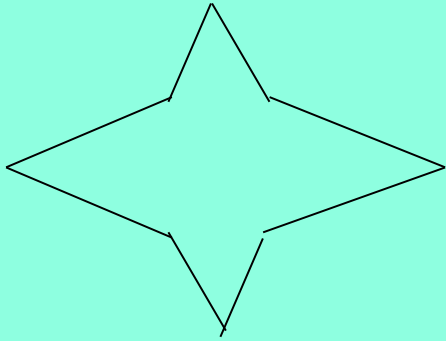
Sutherland-Hodgman

Algorithm

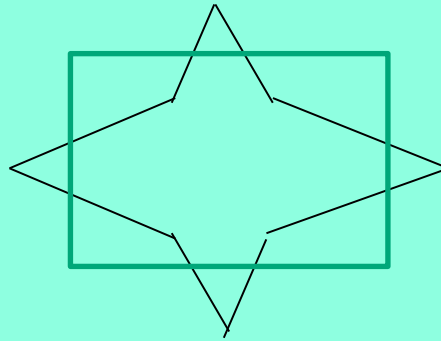
- Basic idea:
 - Consider each edge of the viewport individually
 - Clip the polygon against the edge equation
 - After doing all planes, the polygon is fully clipped

Polygon Clipping

Sutherland-Hodgman Algorithm



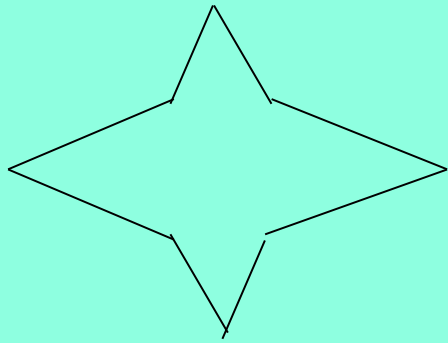
Original polygon



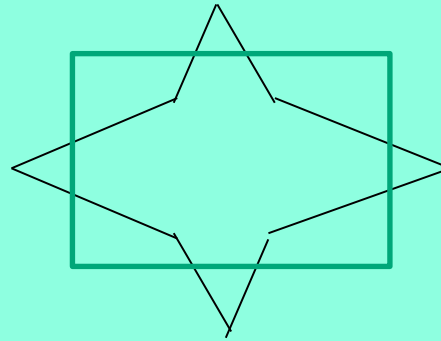
Clipping
Window

Polygon Clipping

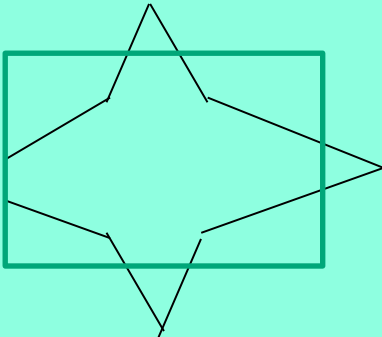
Sutherland-Hodgman Algorithm



Original polygon



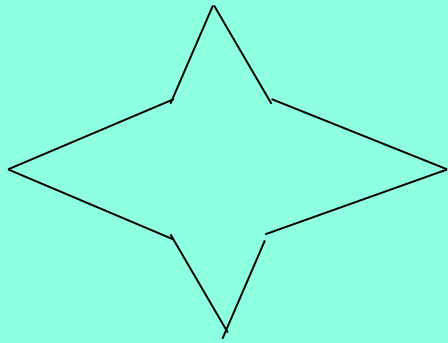
Clipping
Window



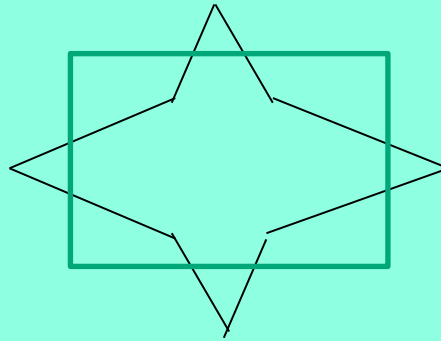
Left
Clip

Polygon Clipping

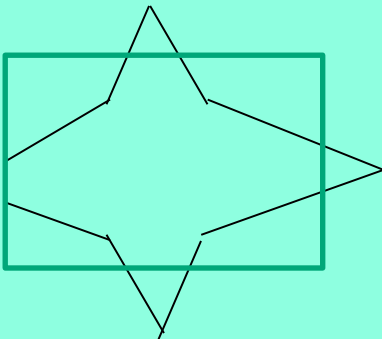
Sutherland-Hodgman Algorithm



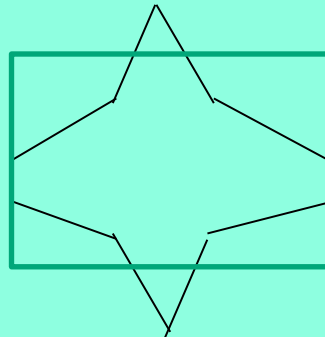
Original polygon



Clipping
Window



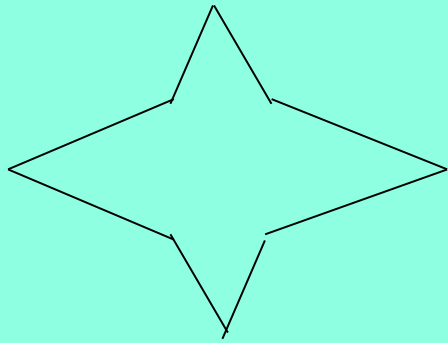
Left
Clip



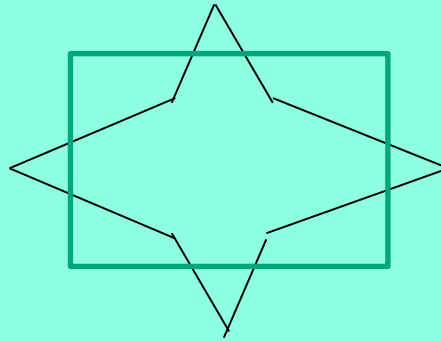
Right
Clip

Polygon Clipping

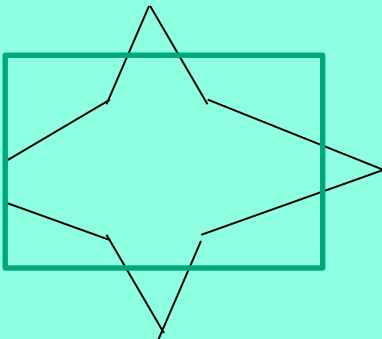
Sutherland-Hodgman Algorithm



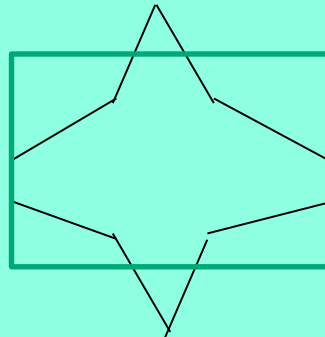
Original polygon



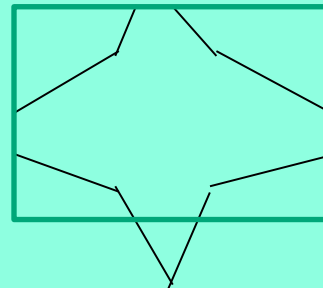
Clipping
Window



Left
Clip



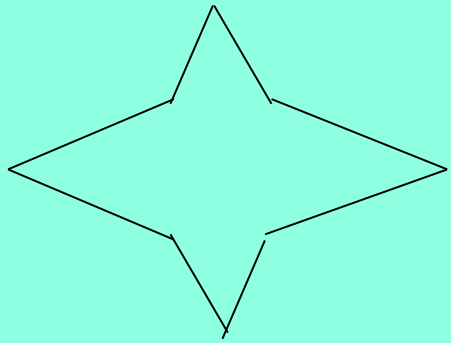
Right
Clip



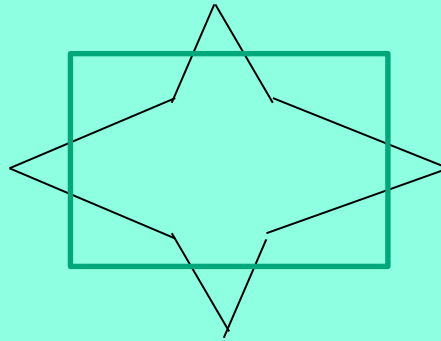
Top Clip

Polygon Clipping

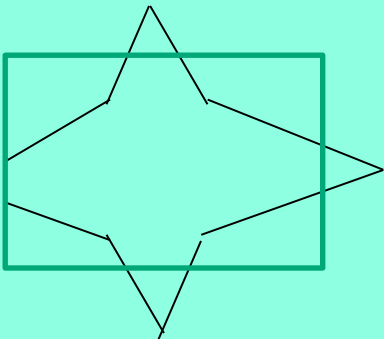
Sutherland-Hodgman Algorithm



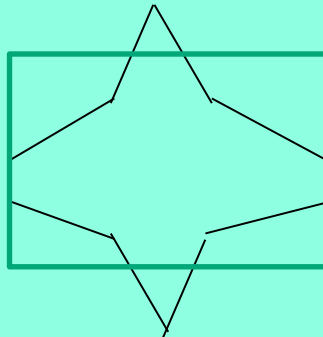
Original polygon



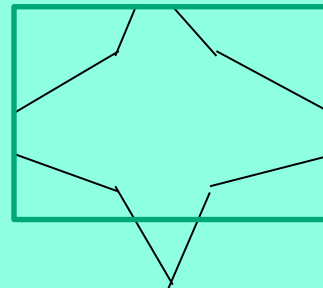
Clipping
Window



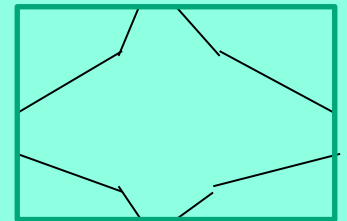
Left
Clip



Right
Clip



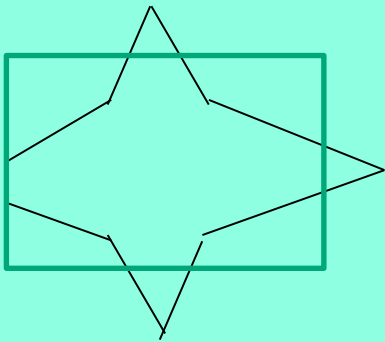
Top Clip



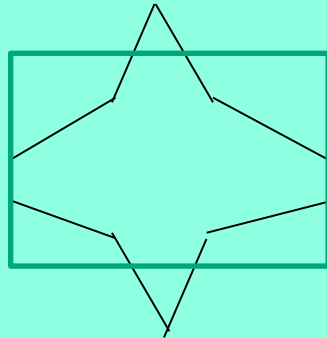
Bottom Clip

Polygon Clipping

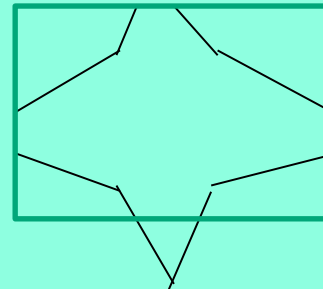
Sutherland-Hodgman Algorithm



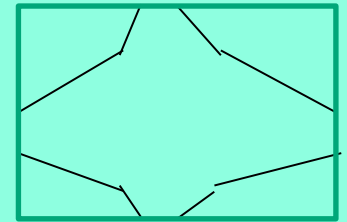
Left
Clip



Right
Clip

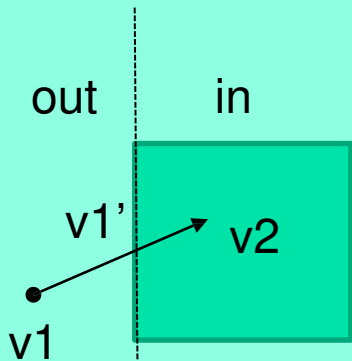


Top Clip

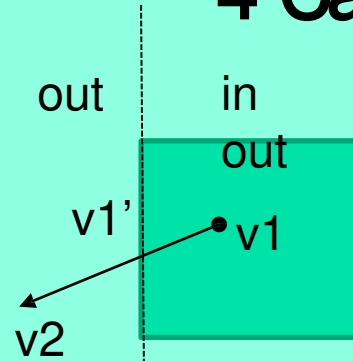


Bottom Clip

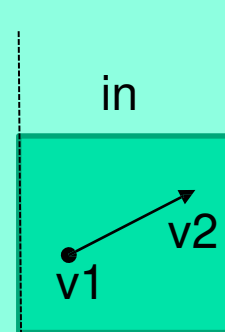
4 Cases



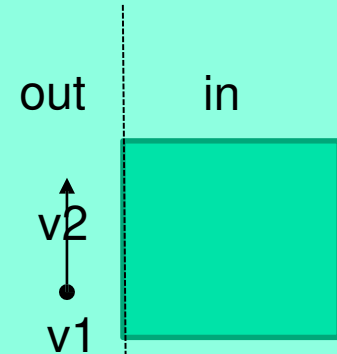
Case 1: Output:
 $(out \rightarrow in) \rightarrow v1'v2$



Case 2: Output:
 $(in \rightarrow out) \rightarrow v1v1'$



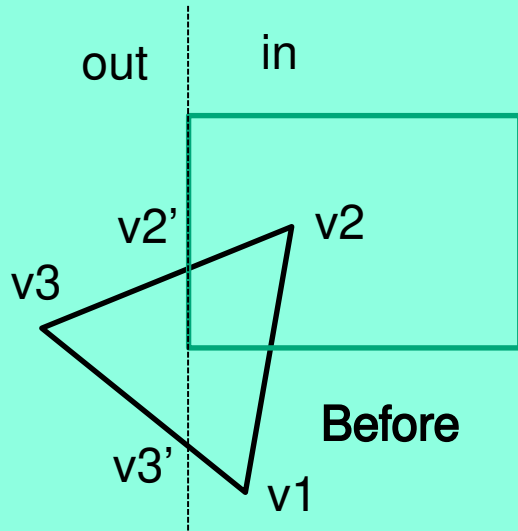
Case 3: Output:
 $(in \rightarrow in) \rightarrow v1v2$



Case 4: Output:
 $(out \rightarrow out) \rightarrow NIL$

Sutherland-Hodgman Algorithm

Example

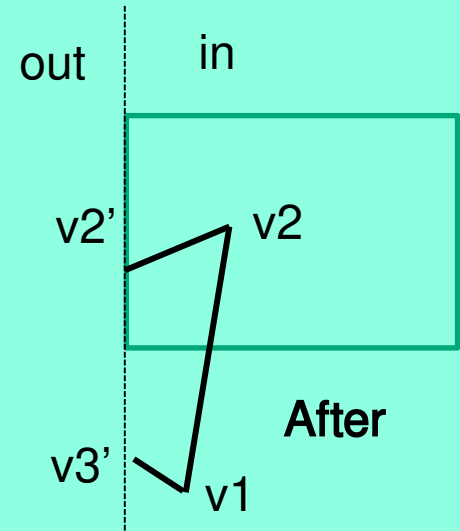


Left Clip

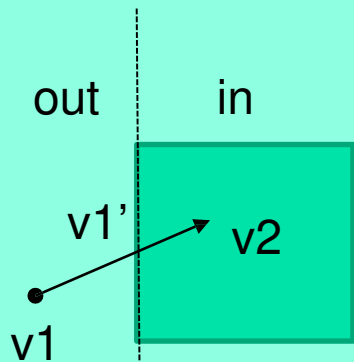
$v1-v2 - (in-in) - v1v2$

$v2-v3 - (in-out) - v2v2'$

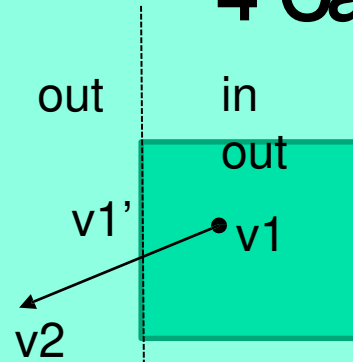
$v3-v1 - (out-in) - v3'v1$



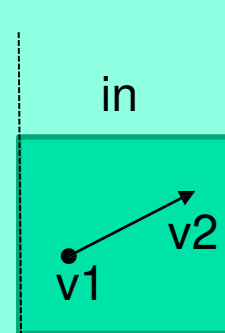
4 Cases



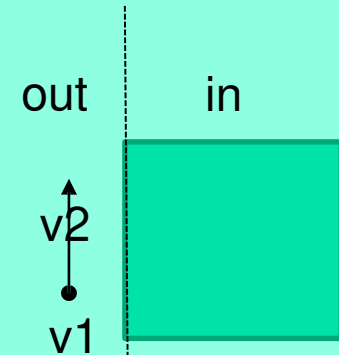
Case 1: Output:
 $(out \rightarrow in) \rightarrow v1'v2$



Case 2: Output:
 $(in \rightarrow out) \rightarrow v1'$



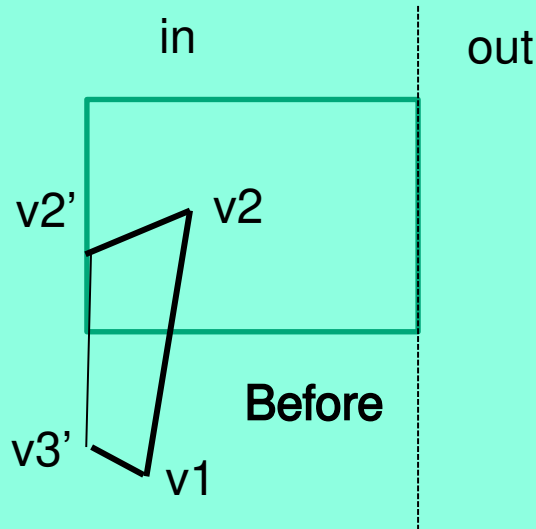
Case 3: Output:
 $(in \rightarrow in) \rightarrow v2$



Case 4: Output:
 $(out \rightarrow out) \rightarrow NIL$

Sutherland-Hodgman Algorithm

Example



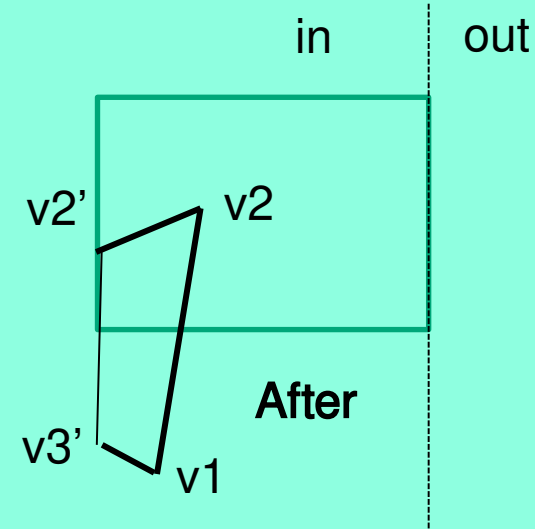
Right Clip

$v1-v2 - (in-in) - v2$

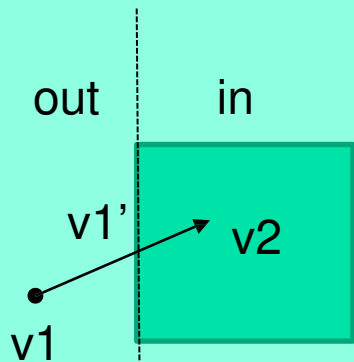
$v2-v2' - (in-in) - v2'$

$v2'-v3' - (in-in) - v3'$

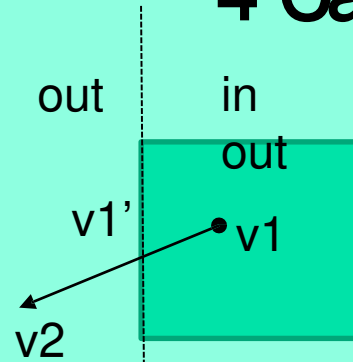
$v3'-v1 - (in-in) - v1$



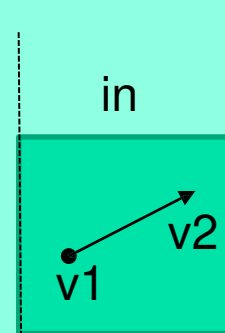
4 Cases



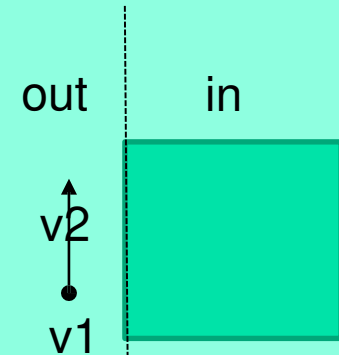
Case 1: Output:
 $(out \rightarrow in) \rightarrow v1'v2$



Case 2: Output:
 $(in \rightarrow out) \rightarrow v1'$



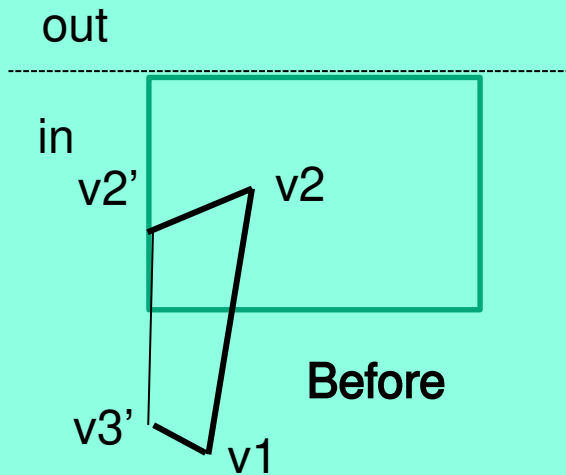
Case 3: Output:
 $(in \rightarrow in) \rightarrow v2$



Case 4: Output:
 $(out \rightarrow out) \rightarrow NIL$

Sutherland-Hodgman Algorithm

Example



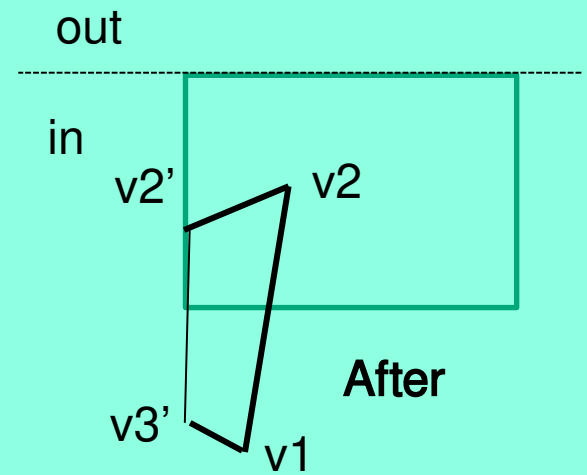
Top Clip

$v1-v2 - (in-in) - v2$

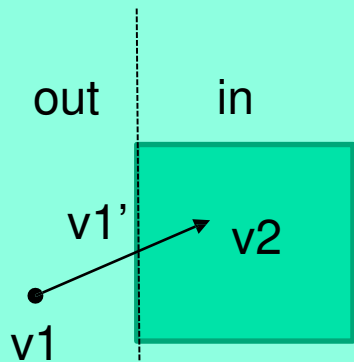
$v2-v2' - (in-in) - v2'$

$v2'-v3' - (in-in) - v3'$

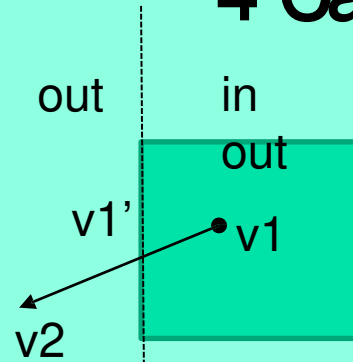
$v3'-v1 - (in-in) - v1$



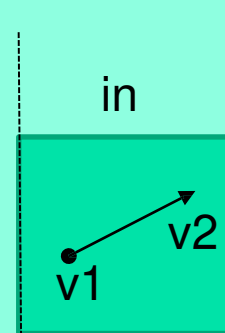
4 Cases



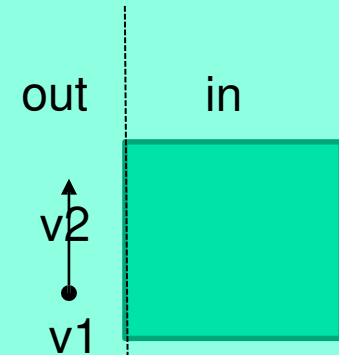
Case 1: Output:
 $(out \rightarrow in) \rightarrow v1'v2$



Case 2: Output:
 $(in \rightarrow out) \rightarrow v1'$



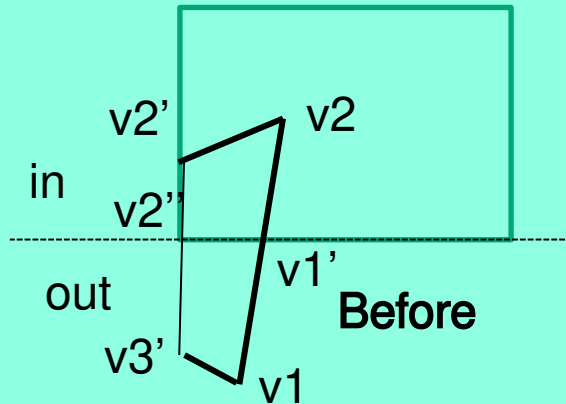
Case 3: Output:
 $(in \rightarrow in) \rightarrow v2$



Case 4: Output:
 $(out \rightarrow out) \rightarrow NIL$

Sutherland-Hodgman Algorithm

Example



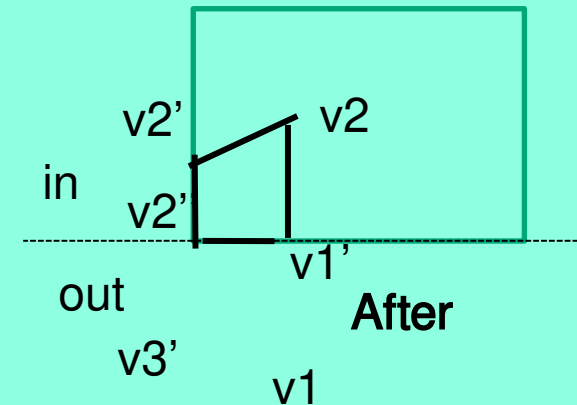
Bottom Clip

$v1-v2 - (out-in) - v1'v2$

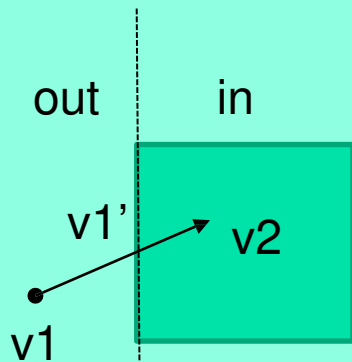
$v2-v2' - (in-in) - v2'$

$v2'-v3' - (in-out) - v2''$

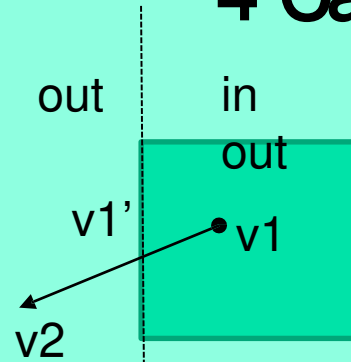
$v3'-v1 - (out-out) - \text{NILL}$



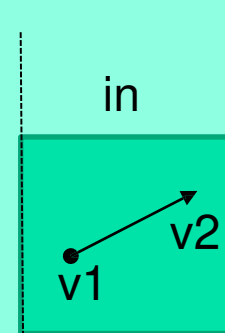
4 Cases



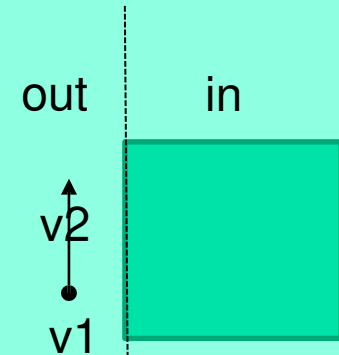
Case 1: Output:
 $(out \rightarrow in) \rightarrow v1'v2$



Case 2: Output:
 $(in \rightarrow out) \rightarrow v1'$



Case 3: Output:
 $(in \rightarrow in) \rightarrow v2$



Case 4: Output:
 $(out \rightarrow out) \rightarrow \text{NIL}$

■ The
END